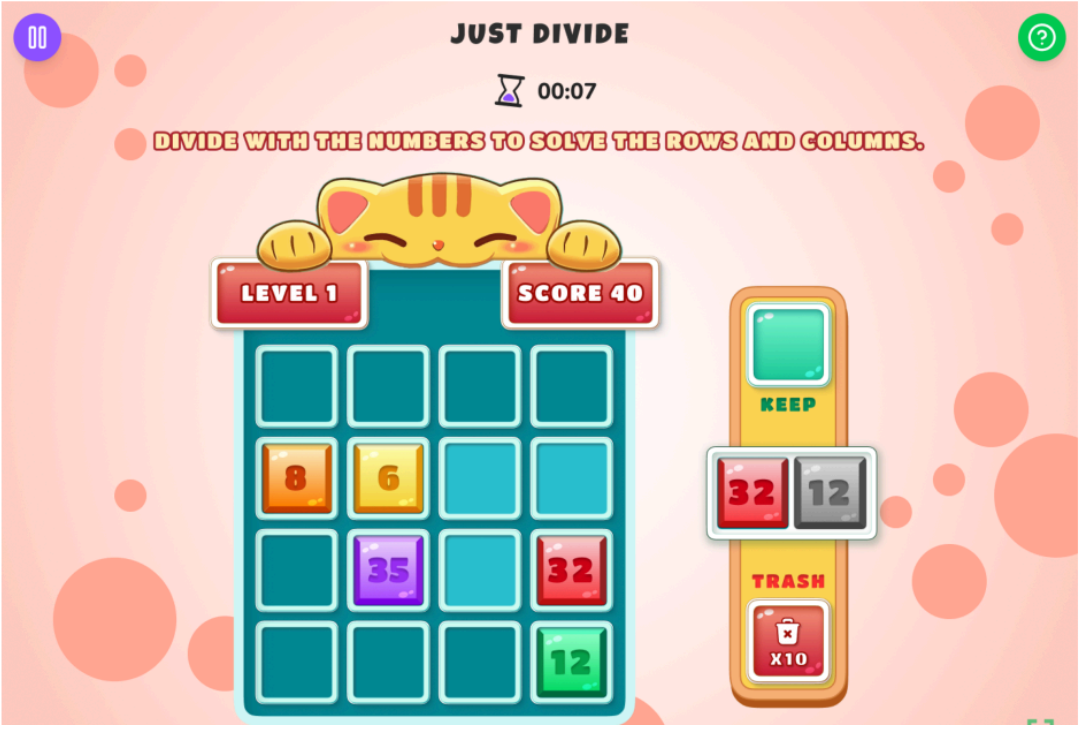
Just Divide - Kid Mode
Game Development Task (ReactJS)

Build a visually faithful, responsive ReactJS implementation of the game described below.

Objective	This task evaluates
Recreate the complete "Just Divide - Kid Mode" game in ReactJS based on the functional and visual requirements in this document.	• UI recreation accuracy • Game logic correctness • Responsive layout handling • Code structure and clarity • Polish and problem-solving

1. Game Overview

Just Divide - Kid Mode is a math-based puzzle game for children aged 7-12. The goal is to help players understand division, factors, multiples, equal values, and simple strategic thinking through short drag-and-drop turns.



Reference visual from the original brief. Use the supplied assets exactly as provided.

How it works

- The player takes the top tile from the upcoming queue and drags it into a 4 x 4 grid.
- When the placed tile touches neighboring tiles, it may merge immediately according to the rules below.
- After resolution, the score updates and the next tile becomes active.

Merge rules

Rule	Condition	Example
Equal tiles disappear	If two touching tiles have the same value, both tiles are removed.	4 next to 4 -> both vanish
Divisible tiles merge	If larger / smaller is a whole number, the result replaces the larger tile and the smaller tile disappears.	12 next to 3 -> 4 15 next to 5 -> 3 9 next to 3 -> 3
Result of 1 is removed	A tile with value 1 is never placed directly; if division leads to 1, remove the resulting tile.	1 may appear only as a quotient

2. Core Gameplay Flow

1. Take the top tile from the upcoming stack.
2. Drag it to a grid cell, the KEEP slot, or the TRASH slot.

3. Resolve merges immediately after placement.

4. Update score, level, trash count, and best score as needed.

5. Continue until the grid is full and no valid merges are possible.

Game Over must show the current score and the best score.

3. UI Layout Requirements

Header area

- Game title: JUST DIVIDE
- Subtitle encouraging players to solve rows and columns by dividing numbers
- Session timer

Cat + badges

- Centered cat above the grid
- Level badge on the left
- Score badge on the right
- Both badges must stay horizontally aligned

Grid

- 4 x 4 layout with 16 turquoise slots
- White borders and even spacing
- Centered within the gameplay area
- Each slot holds at most one tile

Right-side action panel

- KEEP slot stores one tile and supports swapping
- Queue shows 3 upcoming tiles; top tile is draggable
- TRASH discards the active tile with limited uses per level

Background and visual fidelity

- Use the bubble-pattern wallpaper with a pale pink theme.
- Stretch the background to fill the screen with no blank side bars.
- Use the provided PNG/SVG assets exactly as supplied.
- Tile numbers must be centered and clearly legible.

4. Controls, Hints, and State Management

Primary interaction

• Implement drag-and-drop for the active tile.

• The active tile must be draggable to a grid cell, the KEEP slot, or the TRASH slot.

• Touch support should work well on desktop, tablet, and mobile.

Optional keyboard support

• Z -> Undo

• R -> Restart

• 1, 2, 3 -> Difficulty toggle

• G -> Toggle hints

Hint system

When hints are ON, highlight only those empty cells where the current tile could immediately create a valid equality or division-based merge.

Leveling

• Every 10 points advances the player by one level.

• Each level grants new trash uses.

• Difficulty may increase slightly based on your implementation.

Required game state

State variable	Purpose
grid	16 grid cell values
queue	3 upcoming tiles
keepVal	stored tile value
score and bestScore	current score and persisted best score
undoStack	up to 10 previous states
level and trashCount	progress and remaining discards
hintsEnabled and timer	UI assist and session time

5. Technical Requirements

You must use

- ReactJS
- JavaScript
- HTML/CSS
- Responsive scaling
- Design resolution: 1440 x 1024

Implementation expectations

- Use a clean component structure and readable state management.
- Do not use heavyweight game engines or prebuilt gameplay frameworks for the core logic.
- Use localStorage for best score persistence.
- Keep the code lightweight, maintainable, and reviewable.

6. Tasks to Complete

UI tasks

1. Build the full responsive screen using a 1440 x 1024 design space.

2. Position the cat, score/level badges, grid, and right-side panel accurately.

3. Render all supplied SVG/PNG assets correctly.

4. Center text inside tiles.

5. Implement safe scaling across desktop, tablet, and mobile.

Functionality tasks

6. Implement drag-and-drop tile placement.

7. Implement equal-tile and divisible-tile merge logic.

8. Update the queue after each move.

9. Implement KEEP and TRASH mechanics.

10. Support Undo, hints, level-up, and game-over detection.

11. Persist best score in localStorage.

12. Add smooth transitions and interactions.

7. Submission Requirements

Deliverable	Required contents	Notes
GitHub repository	Complete React source code, package.json, README.md, and assets folder	Keep structure clean and easy to review
README.md	Approach, key decisions, challenges, and what you would improve	Concise but specific
Hosted build	Playable deployment on Vercel	Link must be accessible

8. Evaluation Criteria

- Visual accuracy against the brief
- Correctness of merge rules and gameplay flow
- Responsiveness and layout behavior
- Code quality, clarity, and maintainability
- Interaction polish and completeness

AI usage

Keep AI use minimal. You should be able to explain your architecture, logic, and implementation choices clearly during review.

Focus on fidelity, correctness, responsiveness, and clean code.